Projeto 7 – Filtros Digitais
Equalizador de áudio de 12 bandas

Camada Física da Computação – Insper

1. Visão geral

A entrega tem dois scripts Python:
• exercicios.py – resolve os exercícios 1 a 6 (FFT, filtro passa-baixa, decibéis, Bode, fator Q).
• equalizador.py – o Projeto 7: equalizador de 12 bandas (20 Hz a 20 kHz, de -10 a +10 dB) com interface gráfica, reprodução do áudio original e do equalizado e Bode do filtro completo.

Dependências: numpy, scipy, matplotlib e sounddevice (pip install numpy scipy matplotlib sounddevice).
Para rodar: python exercicios.py ou python equalizador.py

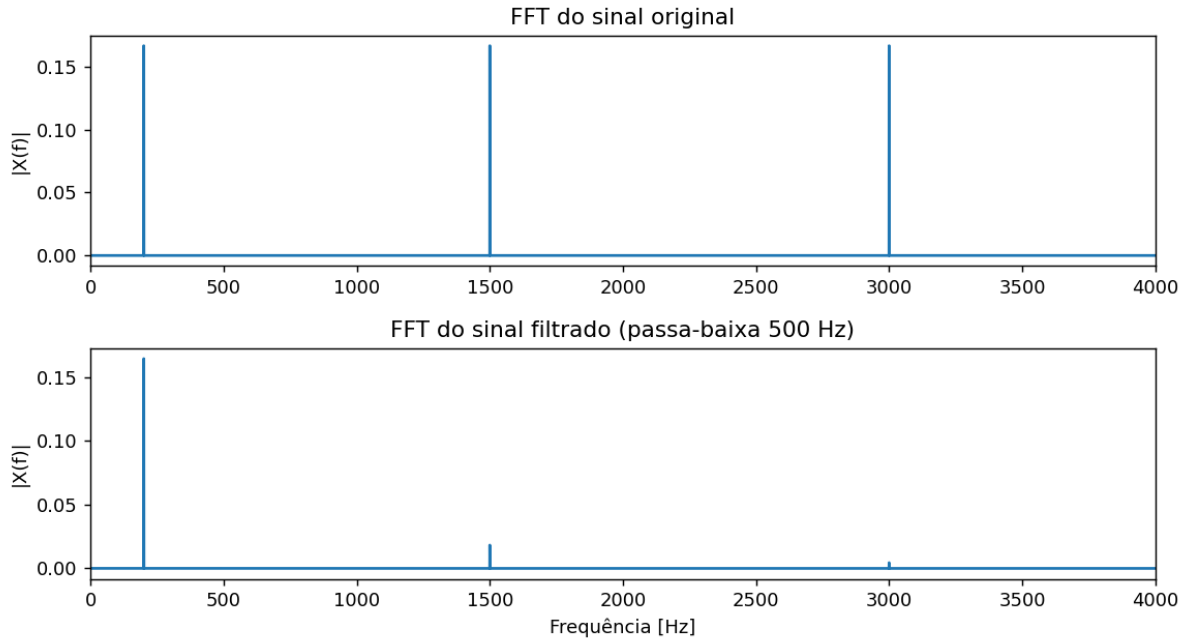
2. Exercícios

Exercício 1 – Passa-baixa de 500 Hz

Geramos 3 s de sinal a 44100 Hz somando senoides de 200 Hz, 1500 Hz e 3000 Hz (as duas últimas acima de 500 Hz). Os coeficientes vêm de butter(N=2, Wn=500/(fs/2)), que são exatamente os do enunciado: b = [0.00120741, 0.00241481, 0.00120741] e a = [1, -1.89933342, 0.90416304].
O filtro foi implementado "na mão" com a equação de diferenças, igual ao exemplo do enunciado:

```
def filtro_ordem2(x, b, a):
    y = [0.0] * len(x)
    for k in range(2, len(x)):
        y[k] = (-a[1] * y[k-1] - a[2] * y[k-2]
                + b[0] * x[k] + b[1] * x[k-1] + b[2] * x[k-2])
    return np.array(y)
```

Os coeficientes de a[] entram com sinal trocado porque, em G(Z), o denominador é Z² + a1·Z + a2: passando para o lado direito da equação, Y[k] = -a1·Y[k-1] - a2·Y[k-2] + ... (a1 = -1,899 vira +1,899·Y[k-1]).
A FFT (np.fft.rfft) mostra os três picos no sinal original; no filtrado, 200 Hz praticamente não muda e 1500 Hz e 3000 Hz caem muito. Ao ouvir, o original soa "agudo/metálico" e o filtrado fica um tom grave e abafado.



Exercício 2 – Limiares em dB

Audição: I_dB = 10·log(10^-12 / 10^-12) = 10·log(1) = 0 dB.
Dor: I_dB = 10·log(10^0 / 10^-12) = 10·log(10^12) = 120 dB.

Exercício 3 – Ganho do filtro

G_dB = 20·log(Vsaída/Ventrada) = 20·log(10/20) = 20·log(0,5) ≈ -6,02 dB.

Exercício 4a – Amplitude de saída pelo Bode

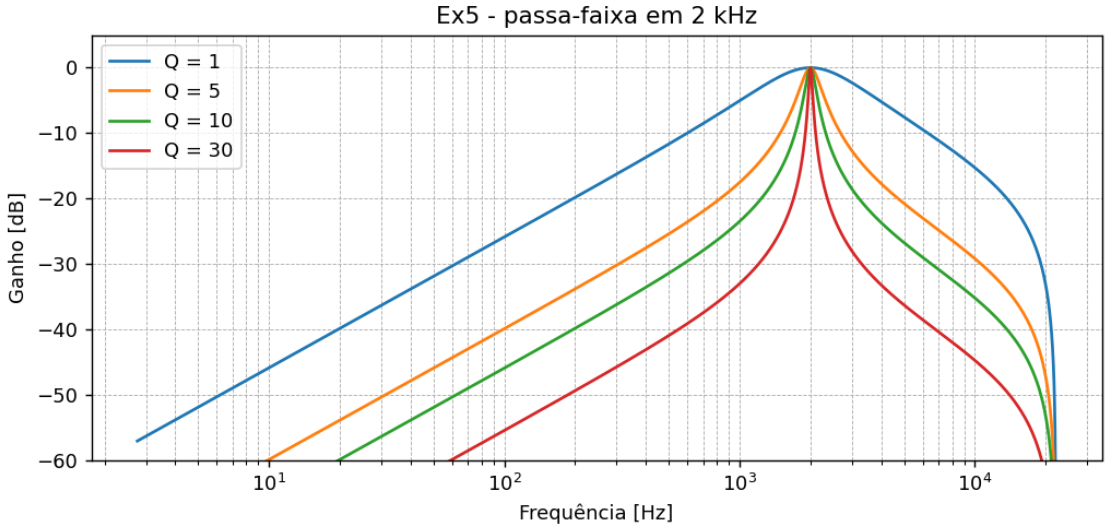
Atenção: no Bode do enunciado o eixo está em rad/s.
• ω = 10^4 rad/s (≈1592 Hz): ganho ≈ -20 dB → Vs/Ve = 10^(-20/20) = 0,1 → amplitude de saída ≈ 50·0,1 = 5.
• ω = 10^3 rad/s (≈159 Hz): ganho ≈ 0 dB → Vs/Ve ≈ 1 → amplitude de saída ≈ 50.
(Conferido no código com freqz: -20,2 dB → 4,9 e -0,0 dB → 49,7.)

Exercício 4b – Frequência de corte (-6 dB)

Pelo gráfico/código, o ganho chega a -6 dB em ω ≈ 4100 rad/s, ou seja, fc ≈ 657 Hz. Nessa frequência Vs/Ve = 10^(-6/20) ≈ 0,5: a saída tem metade da amplitude da entrada.

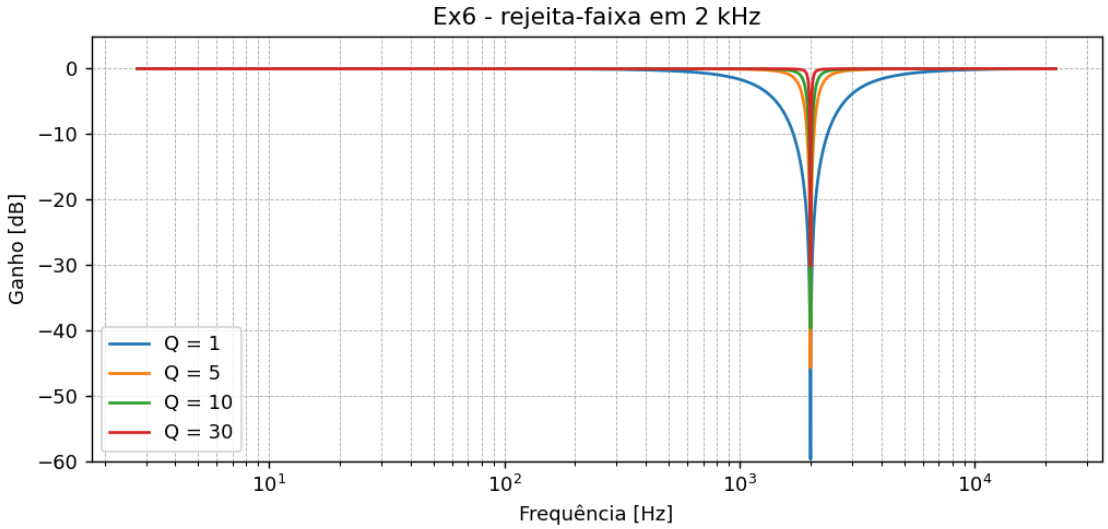
Exercício 5 – Passa-faixa (iirpeak) e o fator Q

Q controla a largura da faixa que passa: largura de banda ≈ f0/Q. Com Q pequeno (1) o filtro deixa passar uma faixa larga em volta de 2 kHz; com Q grande (30) só passa uma faixa estreita bem próxima de 2 kHz. O ganho em f0 continua 0 dB; o que muda é a seletividade.



Exercício 6 – Rejeita-faixa (iirnotch)

É o inverso: Q controla a largura do "buraco". Q alto remove só uma frequência muito estreita; Q baixo remove uma faixa larga. Mudando f0, o buraco se desloca. Utilidade: remover um ruído de frequência fixa sem estragar o resto do sinal, por exemplo o zumbido da rede elétrica de 60 Hz em áudio/ECG, ou um apito (microfonia) num show.



3. Equalizador (equalizador.py)

Fluxo: carrega o áudio de teste → usuário ajusta 12 sliders → botão "Aplicar e tocar" → calcula os 12 filtros, aplica um a um, toca original e equalizado, e plota o Bode do filtro completo.

3.1 Parâmetros

```
FS = 44100
ARQUIVO = "audio_teste.wav"
BANDAS = [20, 32, 64, 125, 250, 500, 1000, 2000, 4000, 8000, 16000, 20000]
Q = 1.4                    # ~1 oitava de largura por banda
```

As bandas são as da figura do enunciado. Elas ficam mais ou menos uma oitava uma da outra (a frequência dobra), por isso Q ≈ 1,4, que é o Q de um filtro com largura de 1 oitava: cada banda cobre o espaço até a vizinha sem sobrepor demais.

3.2 peaking_eq – a função de transferência de cada banda

```
def peaking_eq(f0, gain_db, Q, fs):
    A = 10**(gain_db / 40)
    omega = 2 * np.pi * f0 / fs
    alpha = np.sin(omega) / (2 * Q)
    b = np.array([1 + alpha * A, -2 * np.cos(omega), 1 - alpha * A])
    a = np.array([1 + alpha / A, -2 * np.cos(omega), 1 - alpha / A])
    return b / a[0], a / a[0]
```

É o código do filtro_peaking_EQ fornecido. Ele devolve os coeficientes de um filtro de 2ª ordem:
$G(Z) = (b0 \cdot Z^2 + b1 \cdot Z + b2) / (Z^2 + a1 \cdot Z + a2)$
Em f0 o ganho é exatamente gain_db (positivo amplifica, negativo atenua) e longe de f0 o ganho volta a 0 dB, então cada banda só mexe na sua região. Dividir por a[0] deixa o denominador com coeficiente 1 em Z^2, no mesmo formato do enunciado.

3.3 carrega_audio – áudio de teste

Se o arquivo audio_teste.wav não existir, grava 5 s do microfone com sd.rec e salva. Nas próximas execuções usa sempre o mesmo arquivo (o enunciado pede manter um áudio gravado para testes). Se o .wav for estéreo vira mono, e o sinal é normalizado para [-1, 1]. Dá para trocar audio_teste.wav por qualquer música .wav.

3.4 equaliza – aplica os filtros um a um

```
def equaliza(audio, ganhos, fs):
    filtros = [peaking_eq(f0, g, Q, fs) for f0, g in zip(BANDAS, ganhos)]
    y = audio
    for b, a in filtros:
```

```
        y = lfilter(b, a, y)
    return y, filtros
```

Para cada banda calculamos (b, a) com o ganho escolhido no slider. Depois o sinal passa pelos filtros em série: a saída de um é a entrada do próximo. O lfilter do scipy faz exatamente a mesma equação de diferenças do exercício 1 (conferimos: mesma saída), só que muito mais rápido, o que importa porque são 12 filtros em segundos de áudio.

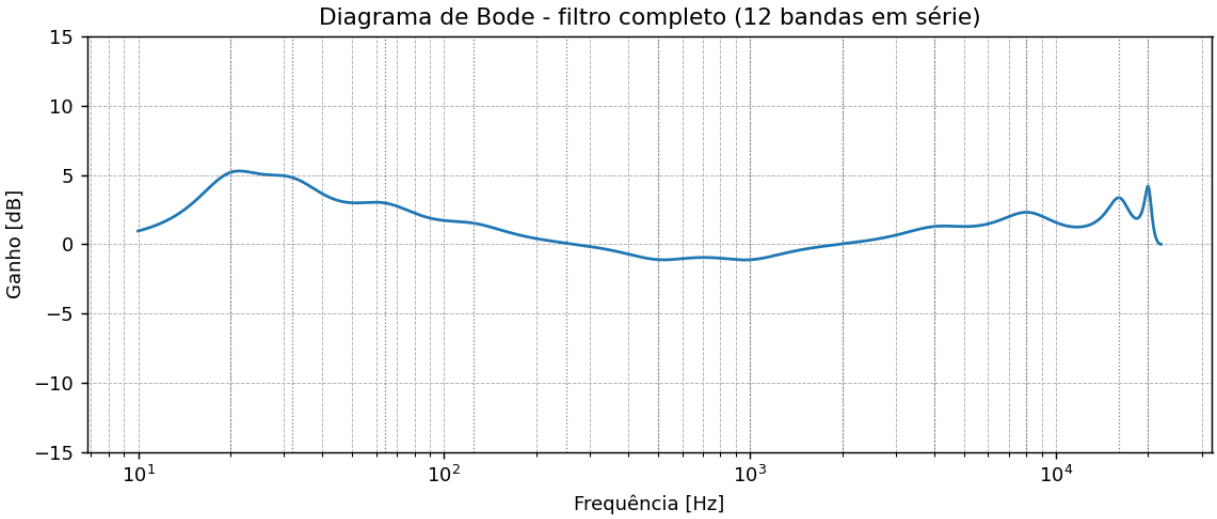
3.5 plota_bode – o filtro como um todo

Filtros em série se multiplicam: G_total(Z) = G1(Z)·G2(Z)·…·G12(Z). Em dB isso vira soma: o ganho total em cada frequência é a soma dos ganhos de cada banda.

A forma direta seria multiplicar os polinômios (np.polymul) e chegar num único G(Z) de grau 24. Testamos e não funciona bem: os polos das bandas graves (20 Hz) ficam colados em Z = 1 e, ao expandir o polinômio de grau 24, o erro de arredondamento do computador joga polos para fora do círculo unitário (filtro instável e Bode errado). Por isso calculamos G_total avaliando cada Gi na frequência (freqz) e multiplicando os resultados, que é a mesma função de transferência total, só que numericamente correta:

```
f = np.logspace(np.log10(10), np.log10(fs / 2), 2000)
H = np.ones(len(f), dtype=complex)
for b, a in filtros:
    H *= freqz(b, a, worN=f, fs=fs)[1]
plt.semilogx(f, 20 * np.log10(np.abs(H)))
```

Exemplo do Bode para a configuração da figura do enunciado (+4, +3, +2, +1, 0, -1, -1, 0, +1, +2, +3, +4 dB):

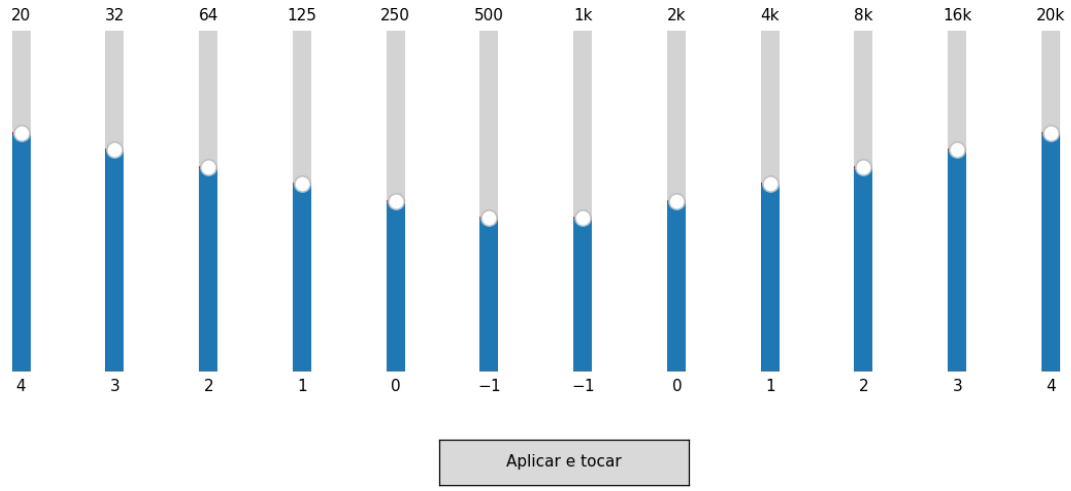


3.6 main – interface com o usuário

A interface usa os widgets do próprio matplotlib (sem biblioteca extra): 12 sliders verticais, um por banda, de -10 a +10 dB em passos de 1 dB, parecidos com um equalizador de verdade, e um botão. Ao clicar em "Aplicar e tocar":

1. lê os 12 ganhos dos sliders;
2. chama equaliza (filtros aplicados ao sinal);
3. se o resultado passar de 1 em módulo (amplificação), normaliza para não distorcer (clipping);
4. toca o original e, em seguida, o equalizado (sd.play + sd.wait);
5. abre o Bode do filtro completo.

Dá para mexer nos sliders e clicar de novo quantas vezes quiser.



3.7 Validação

Com um seno puro em cada banda e só aquela banda alterada, o ganho medido na saída foi exatamente o configurado: 1 kHz com +10 dB → +10,0 dB; 64 Hz com -10 dB → -10,0 dB; 16 kHz com +6 dB → +6,0 dB.